

**DESIGN AND HARDWARE IMPLEMENTATION OF A
32-BIT RISC-V PROCESSOR USING VERILOG HDL
ON ARTY A7 FPGA**

A PROJECT REPORT

Submitted by

KIRANKUMAR A

in partial fulfillment of the requirements for the award of the Degree

of

Post Graduate Program

In

VLSI SoC Design and Verification



National Institute of Electronics and Information Technology (NIELIT)

Calicut

July 2026

DECLARATION

I undersigned hereby declare that the project report "**DESIGN AND HARDWARE IMPLEMENTATION OF A 32-BIT RISC-V PROCESSOR USING VERILOG HDL ON ARTY A7 FPGA**" submitted for partial fulfillment of the requirements for the award of the degree of PG Program in VLSI SoC Design and Verification is a bonafide work done by me under the supervision of **Shri. Sreejeesh S G (STO, NIELIT, Calicut) and Nanditha N Varma(Adhoc Faculty)**. This submission represents my ideas in my own words and where ideas or words of others have been included, I have adequately and accurately cited and referenced the original sources. I also declare that I have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed as the basis for the award of any degree, diploma, or similar title of any other University.

Place: NIELIT, Calicut

Date: 13-07-2026



Kirankumar A

DEPARTMENT OF EMBEDDED SYSTEMS

**National Institute of Electronics and Information Technology
(NIELIT) Calicut**



CERTIFICATE

This is to certify that the report entitled, **"DESIGN AND HARDWARE IMPLEMENTATION OF A 32-BIT RISC-V PROCESSOR USING VERILOG HDL ON ARTY A7 FPGA"** submitted by **Mr. KIRANKUMAR A** in partial fulfillment of the requirements for the award of the PG Program in VLSI SoC Design and Verification is a Bonafide record of the project work carried out by him under my guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Internal Supervisor(s)

Sreejeesh S G
Senior Technical Officer
NIELIT Calicut

Nanditha N Varma
Adhoc Faculty
NIELIT Calicut

CONTENTS

ACKNOWLEDGEMENT.....	i
ABSTRACT.....	ii
LIST OF TABLES	iii
LIST OF FIGURES	iv
ABBREVIATIONS	v
1. INTRODUCTION.....	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objective	2
1.4 Organisation of the Project	2
2. LITERATURE SURVEY.....	4
2.1 Review of Related Research and Projects	4
3. OVERVIEW.....	6
3.1 RISC-V Instruction Set Architecture	6
3.2 Processor Execution Models	7
4. DESIGN	8
4.1 Block Diagram of the Processor	8
4.2 Module Descriptions	8
5. IMPLEMENTATION	12
5.1 Bit-Width and Memory Specifications	12
5.2 I/O Ports	12
5.3 Test Program	13
5.4 Simulation Setup	14
5.5 Hardware Implementation Setup	14
5.6 Synthesis and Implementation Reports	15
6. RESULT AND DISCUSSION	17
6.1 Simulation Results	17
6.2 Hardware Results	19
6.3 Discussion	21
7. CONCLUSION AND FUTURE SCOPE	22
7.1 Conclusion	22
7.2 Future Scope	22
REFERENCES.....	24

ACKNOWLEDGEMENT

I express my sincere gratitude to all the people who have extended their help and support towards the successful completion of my project work.

I am thankful to **Dr. Jayaraj U. Kidav**, Director, National Institute of Electronics and Information Technology, Calicut for providing the necessary facilities for the completion of my course.

I am highly thankful to **Shri. Sreejeesh S G**, STO, NIELIT, Calicut, for his suggestions and guidance during the course of this project.

I heartily thank **Nanditha N Varma** , Adhoc Faculty, for her constant feedback and suggestions, and proper guidance for the progress of the project.

Last but not least, I express my thanks to my family and friends who have supported me throughout, and I thank Almighty for all the blessings.

ABSTRACT

RISC-V is a free and open-standard Instruction Set Architecture (ISA) that has emerged as a popular alternative to proprietary architectures for academic, research and embedded system applications. This project presents the design, simulation and hardware implementation of a 32-bit single-cycle processor based on the RV32I base integer instruction set, described entirely in Verilog HDL.

The processor was built from individual modules — Program Counter, Instruction Memory, Register File, Immediate Generator, Control Unit, Arithmetic Logic Unit (ALU), Data Memory and the associated multiplexers — each verified independently with dedicated testbenches before being integrated into a complete single-cycle datapath. A custom test program covering all major RV32I opcodes (R-type, I-type, load, store and jump instructions) was hand-assembled and used to functionally verify the processor in Vivado behavioral simulation, achieving correct results for all opcodes.

The design was then made synthesizable and implemented on a Xilinx Arty A7-100T FPGA (xc7a100tcsg324-1) through a remote hardware laboratory. Since the remote setup does not provide access to the physical reset button, a Virtual Input/Output (VIO) IP core was integrated into the hardware top module to control the processor's reset signal and to observe the ALU result and decoded opcode directly from the FPGA. A clock divider was added to slow the 100 MHz system clock so that the on-board LEDs visibly display the result of each executed instruction. The bitstream was generated, the FPGA was programmed over the remote lab connection, and the processor was verified to correctly execute all opcodes on real hardware, with the LED outputs and VIO dashboard values confirmed to match the expected results from simulation.

This project demonstrates the complete journey of a processor design — from RTL coding and module-level verification, through full-system simulation, to real FPGA hardware deployment and remote verification — and serves as a practical foundation for understanding computer architecture and digital design flow.

NIELIT, Calicut

LIST OF TABLES

No.	Title	Page No.
3.1	RV32I base instruction formats	6
4.1	Opcode to control signal mapping	9
4.2	ALU control (ALUCtrl) encoding	10
5.1	Bit-width and memory specification	12
5.2	I/O ports of the hardware top module	12
6.1	Simulation verification summary (14 opcodes)	17

LIST OF FIGURES

No.	Title	Page No.
4.1	Block diagram of the RISC-V processor	8
5.1	Resource utilization summary	16
5.2	Timing summary	16
5.3	Power utilization Summary	16
6.1	Simulation waveform	18
6.2	Tcl Console showing simulation PASS results	18
6.3	Vivado Hardware Manager – bitstream programming	19
6.4	Physical board with LED output glowing	20
6. 5	VIO dashboard showing led_result and led_opcode	20

ABBREVIATIONS

ISA	Instruction Set Architecture
RV32I	RISC-V 32-bit Integer Base Instruction Set
HDL	Hardware Description Language
FPGA	Field Programmable Gate Array
ALU	Arithmetic Logic Unit
PC	Program Counter
RF	Register File
IMEM	Instruction Memory
DMEM	Data Memory
MUX	Multiplexer
RTL	Register Transfer Level
VIO	Virtual Input/Output (Xilinx debug IP core)
ILA	Integrated Logic Analyzer
XDC	Xilinx Design Constraints
LUT	Look-Up Table
I/O	Input-Output

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Computer architecture forms the foundation of every digital computing system, and the processor is its most essential component. For decades, processor design was dominated by proprietary instruction set architectures such as x86 and ARM, which restricted academic study and low-cost hardware experimentation. RISC-V, first developed at the University of California, Berkeley, was introduced as a free and open-standard Instruction Set Architecture (ISA) that anyone can implement, study, or modify without licensing restrictions.

RISC-V follows Reduced Instruction Set Computer (RISC) design principles — a small set of simple, fixed-length instructions, a large uniform register file, and a load-store memory model. Its openness and simplicity have made it a popular choice for academic processor-design courses, embedded systems and increasingly for commercial System-on-Chip designs. The RV32I base integer instruction set, used in this project, defines 32-bit instructions and 32 general-purpose registers, and is sufficiently complete to execute real programs while remaining simple enough to implement and verify within an academic project timeline.

Designing a processor from RTL coding through simulation and finally to real FPGA hardware gives practical insight into every stage of the digital design flow — something that is not fully appreciated by studying architecture only in theory.

1.2 PROBLEM STATEMENT

While RISC-V soft-core implementations are widely available as open-source repositories, simply reusing an existing core does not build a genuine understanding of processor internals. There is a need for a processor design that is built and verified module-by-module from first principles, so that every control signal, datapath connection and opcode decode can be explained and justified by the designer.

In addition, most academic RISC-V mini-projects stop at behavioral simulation and do not carry the design through to a working hardware demonstration, which leaves an important part of the design flow — synthesis, timing closure, pin-constraint mapping and on-board verification — unexplored.

1.3 OBJECTIVE

The primary objective of this project is to design, simulate and implement a 32-bit single-cycle RISC-V (RV32I) processor using Verilog HDL, and to verify its correct operation both through Vivado behavioral simulation and through real hardware execution on a Xilinx Arty A7-100T FPGA. Specific objectives include:

- Design each functional module of the processor (Program Counter, Instruction Memory, Register File, Immediate Generator, Control Unit, ALU, Data Memory) independently in Verilog HDL.
- Verify each module using dedicated testbenches before integrating them into a complete single-cycle datapath.
- Assemble a custom test program covering all major RV32I opcodes and verify the integrated processor through behavioral simulation.
- Make the design synthesizable and implement it on the Arty A7-100T FPGA through a remote hardware laboratory.
- Use a Virtual Input/Output (VIO) core to control reset and observe processor outputs remotely, and confirm opcode execution using on-board LEDs.

1.4 ORGANISATION OF THE PROJECT

CHAPTER 1: INTRODUCTION – presents the background, problem statement and objectives of the project.

CHAPTER 2: LITERATURE SURVEY – reviews existing research and projects related to RISC-V processor design and FPGA implementation.

CHAPTER 3: OVERVIEW – discusses the RISC-V instruction set architecture, instruction formats and processor execution models.

CHAPTER 4: DESIGN – explains the architectural design of the processor, including the block diagram and description of each module.

CHAPTER 5: IMPLEMENTATION – details the Verilog implementation, bit-width specifications, I/O ports, simulation setup and hardware implementation setup.

CHAPTER 6: RESULT AND DISCUSSION – presents the simulation and hardware results along with a discussion of the findings.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE – summarizes the project and suggests possible future improvements.

CHAPTER 2

LITERATURE SURVEY

2.1 REVIEW OF RELATED RESEARCH AND PROJECTS

1. Implementation of RISC-V Single Cycle Core

Author(s): published in IJARCCCE (2025)

This paper presents the design and implementation of a 32-bit single-cycle RV32I processor using Verilog HDL, targeting FPGA deployment for educational applications. It describes the standard fetch-decode-execute-memory-writeback stages, and discusses a five-stage pipelined extension of the same design. The processor was synthesized on Xilinx Spartan-3E/Spartan-6 boards and verified using Vivado simulation, reporting a maximum operating frequency of about 32 MHz. This work is directly relevant as it validates the feasibility of a fully synthesizable, low-cost, single-cycle RV32I processor of the same scale as the one designed in this project.

2. Single Cycle RISC-V Microarchitecture Processor and its FPGA Prototype

IEEE Conference Publication

This paper describes a fully synthesizable 32-bit single-core, in-order RISC-V (RV32I) processor with low hardware complexity, aimed at low-cost embedded devices. The authors also present a development and validation framework including assembling tools and automated test suites, and prototype the design on a Spartan-3E FPGA. The emphasis on a lightweight test framework and low hardware complexity closely matches the approach followed in the present project, where a hand-assembled test program was used to verify the processor before hardware deployment.

3. Design and FPGA Implementation of Five Stage Pipelined RISC-V Processor

IEEE Conference Publication

This paper implements both a single-cycle and a five-stage pipelined RV32I processor on a Basys-3 FPGA board, reporting a maximum operating frequency of 31.6 MHz for the single-cycle version against 87.86 MHz for the pipelined version.

The comparison highlights the throughput benefit of pipelining at the cost of additional hazard-handling complexity. This provided useful context for understanding why a single-cycle architecture, while simpler to design and verify — as chosen for this project — trades off maximum clock frequency compared to a pipelined design.

4. Design and Implementation of a RISC-V Processor on FPGA

IEEE Conference Publication

This paper focuses specifically on creating a lightweight, beginner-friendly RISC-V processor implementation using modern hardware design techniques, implemented and tested on an FPGA. The authors note that although several open-source RISC-V cores exist, few are written to be easily understood by newcomers to processor design. This motivation directly aligns with the approach of the present project, where every module was written and verified from scratch specifically to build a first-hand understanding of processor internals, rather than reusing an existing core.

CHAPTER 3 OVERVIEW

3.1 RISC-V INSTRUCTION SET ARCHITECTURE

RISC-V defines a family of related ISAs, ranging from a small base integer instruction set to several optional standard extensions such as multiplication/division (M), atomics (A), single and double precision floating point (F, D), and compressed instructions (C). Each processor implementation chooses a base ISA and any extensions it wishes to support. This project implements only the mandatory base integer instruction set, RV32I, which is sufficient to execute general-purpose integer programs.

3.1.1 RV32I Base Integer Instruction Set

RV32I defines 32-bit wide instructions and 32 general-purpose registers (x0 to x31), each 32 bits wide. Register x0 is hardwired to the constant value zero. The instruction set includes arithmetic and logical operations (ADD, SUB, AND, OR, XOR, shifts), comparison operations (SLT, SLTU), memory access operations (LW, SW), and control transfer operations (BEQ, JAL, JALR). Every instruction, regardless of type, is decoded from a fixed set of instruction fields — opcode, funct3, funct7, rd, rs1 and rs2 — which greatly simplifies decode-logic design compared to variable-length instruction sets.

3.1.2 Instruction Formats

RV32I instructions are grouped into six formats depending on how their fields are arranged. Table 3.1 summarizes the six instruction formats used in this project.

Format	Used for	Field layout
R-type	Register-register ops (ADD, SUB, AND, OR, XOR, SLT, SLTU, shifts)	funct7 rs2 rs1 funct3 rd opcode
I-type	Immediate ops and loads (ADDI, SLLI, SRLI, LW)	imm[11:0] rs1 funct3 rd opcode
S-type	Store instructions (SW)	imm[11:5] rs2 rs1 funct3 imm[4:0] opcode
B-type	Conditional branches (BEQ)	imm[12,10:5] rs2 rs1 funct3 imm[4:1,11] opcode
U-type	Upper immediate (LUI, AUIPC)	imm[31:12] rd opcode

Format	Used for	Field layout
J-type	Unconditional jump (JAL)	imm[20,10:1,11,19:12] rd opcode

Table 3.1 RV32I base instruction formats

3.2 PROCESSOR EXECUTION MODELS

A processor can be designed using different execution models that trade off design simplicity against performance. Two common models are described below.

3.2.1 Single-Cycle Architecture

In a single-cycle processor, every instruction — regardless of its type — completes all five conceptual stages (fetch, decode, execute, memory access, write-back) within one clock cycle. This makes the control logic straightforward, since there is no need to hold partial results between cycles or to detect hazards between overlapping instructions. The clock period must, however, be long enough to accommodate the slowest instruction's complete datapath delay, which limits the maximum operating frequency. This project adopts the single-cycle architecture for its conceptual clarity and ease of verification, which is well suited to an academic implementation and hardware demonstration.

3.2.2 Multi-Cycle and Pipelined Architectures

Multi-cycle processors split instruction execution across several clock cycles, allowing each cycle to be shorter and enabling reuse of hardware resources (for example, a single adder used for both address calculation and ALU operations across different cycles). Pipelined processors go further by overlapping the execution of multiple instructions, with each instruction occupying a different pipeline stage on every clock cycle. This significantly improves instruction throughput, at the cost of additional control complexity to detect and resolve data and control hazards between overlapping instructions. These architectures were considered during the design phase but were not adopted for this project, in favor of the single-cycle model described above.

CHAPTER 4 DESIGN

4.1 BLOCK DIAGRAM OF THE PROCESSOR

The complete single-cycle datapath of the processor is shown in Figure 4.1. The Program Counter supplies the fetch address to the Instruction Memory, which outputs the 32-bit instruction. The instruction fields feed the Control Unit (which generates all control signals), the Register File (for reading source operands) and the Immediate Generator (for sign-extending the immediate field). The ALU computes the required operation using either two register operands or a register and an immediate, selected by the ALUSrc multiplexer. The ALU result is used either directly, or as an address into Data Memory for load/store instructions. The value written back to the register file is selected among the ALU result, memory read data, or PC+4, by the write-back multiplexer. The next PC value is selected among PC+4, a branch target, or a jump target, by the PC-next multiplexer.

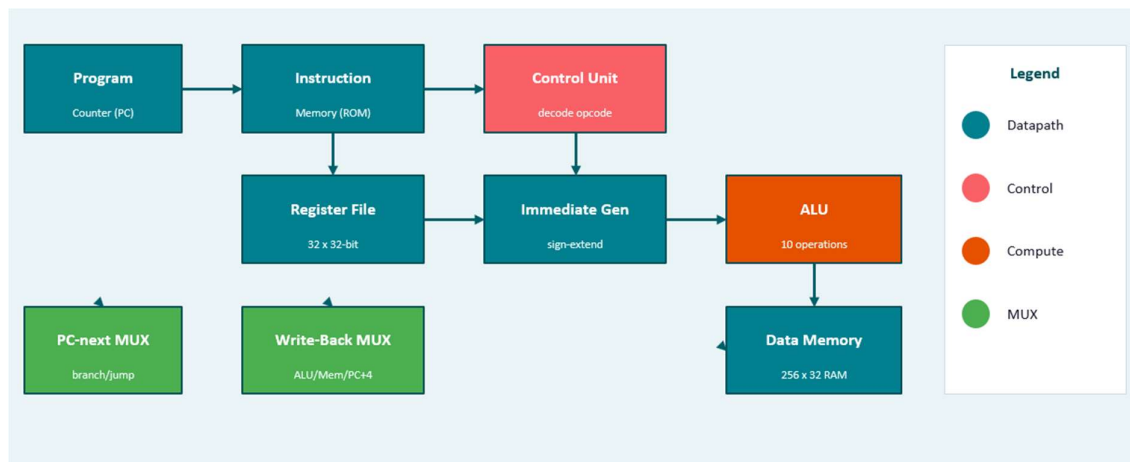


Figure 4.1 Block diagram of the RISC-V processor

4.2 MODULE DESCRIPTIONS

4.2.1 Program Counter

The Program Counter (PC) is a 32-bit register that holds the address of the instruction currently being fetched. On every rising clock edge it is updated to the next PC value computed by the PC-next multiplexer, unless the reset signal is asserted, in which case it is set to address zero.

4.2.2 Instruction Memory

The Instruction Memory is a 256 x 32-bit read-only memory, addressed by the Program Counter, that stores the machine code of the test program. It is implemented as a combinational read — the instruction corresponding to the current PC is available continuously, without waiting for a clock edge.

4.2.3 Register File

The Register File contains 32 general-purpose 32-bit registers (x0 to x31), with register x0 hardwired to zero. It supports two asynchronous (combinational) read ports for the two source registers, and one synchronous write port, updated on the rising clock edge when the RegWrite control signal is asserted.

4.2.4 Immediate Generator

The Immediate Generator extracts and sign-extends the immediate field embedded in the instruction, according to the instruction format (I, S, B, U or J type) determined from the opcode. This produces a uniform 32-bit sign-extended immediate value for use by the ALU or for branch/jump target calculation.

4.2.5 Control Unit

The Control Unit is a purely combinational block that decodes the opcode, funct3 and funct7 fields of the current instruction and generates all control signals required by the rest of the datapath: RegWrite, ALUSrc, MemRead, MemWrite, MemToReg, Branch, Jump, and the 4-bit ALUCtrl signal that selects the ALU operation. Table 4.1 summarizes the control signals generated for each opcode.

Opcode	RegW	ALUSrc	MemRd	MemWr	M2Reg	Branch	Jump
R-type (0110011)	1	0	0	0	0	0	0
I-type (0010011)	1	1	0	0	0	0	0
LW (0000011)	1	1	1	0	1	0	0
SW (0100011)	0	1	0	1	0	0	0

Opcode	RegW	ALUSrc	MemRd	MemWr	M2Reg	Branch	Jump
BEQ (1100011)	0	0	0	0	0	1	0
JAL (1101111)	1	0	0	0	0	0	1

Table 4.1 Opcode to control signal mapping

4.2.6 Arithmetic Logic Unit (ALU)

The ALU performs all arithmetic, logical, shift and comparison operations required by the instruction set. It accepts two 32-bit operands and a 4-bit ALUCtrl selector, and outputs a 32-bit result along with a Zero flag (asserted when the result equals zero), which is used for branch-condition evaluation. Table 4.2 lists the ALU control encoding implemented.

ALUCtrl	Operation
0000	ADD (addition)
0001	SUB (subtraction)
0010	AND (bitwise AND)
0011	OR (bitwise OR)
0100	XOR (bitwise XOR)
0101	SLL (shift left logical)
0110	SRL (shift right logical)
0111	SRA (shift right arithmetic)
1000	SLT (set less than, signed)
1001	SLTU (set less than, unsigned)

Table 4.2 ALU control (ALUCtrl) encoding

4.2.7 Data Memory

The Data Memory is a 256 x 32-bit read/write memory used by the LW and SW instructions. Writes are synchronous (performed on the rising clock edge when MemWrite is asserted), while reads are combinational, so that the loaded value is available within the same cycle for the write-back stage.

4.2.8 Multiplexers

Three multiplexers coordinate data selection in the datapath: the ALUSrc multiplexer selects between the second register operand and the sign-extended immediate as the ALU's second input; the MemToReg (write-back) multiplexer selects among the ALU result, memory read data, and PC+4 as the value written back to the register file; and the PC-next multiplexer selects among PC+4, the branch target address, and the jump target address as the next Program Counter value.

CHAPTER 5

IMPLEMENTATION

5.1 BIT-WIDTH AND MEMORY SPECIFICATIONS

The following width and memory specifications were used throughout the design:

Parameter	Specification
Data width	32 bits
Instruction width	32 bits
Number of registers	32 (x0 – x31), x0 hardwired to 0
Instruction memory size	256 x 32-bit words
Data memory size	256 x 32-bit words
ALU operations supported	10 (ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU)

Table 5.1 Bit-width and memory specification

5.2 I/O PORTS

The hardware top module exposes a minimal set of I/O ports so that it fits within the pin budget of the Arty A7-100T board while still allowing full observability of processor execution through the on-board LEDs. Table 5.2 lists the ports of the hardware top module.

Port	Direction	Description
clk	Input	100 MHz system clock from the on-board oscillator
led_result[7:0]	Output	Lower 8 bits of the ALU result, displayed on 8 LEDs
led_opcode[3:0]	Output	Decoded instruction type, displayed on 4 LEDs

Table 5.2 I/O ports of the hardware top module

The reset signal (rst) is not a top-level port in the hardware version; instead, it is driven internally by the Virtual Input/Output (VIO) core, as described in section 5.5.3.

5.3 TEST PROGRAM

- ADDI x1, x0, 5 (x1 = 5)
- ADDI x2, x0, 10 (x2 = 10)
- ADD x3, x1, x2 (x3 = 15)
- SUB x4, x2, x1 (x4 = 5)
- AND x5, x1, x2 (x5 = 0)
- OR x6, x1, x2 (x6 = 15)
- XOR x7, x1, x2 (x7 = 15)
- SLLI x8, x1, 1 (x8 = 10)
- SRLI x9, x2, 1 (x9 = 5)
- SLT x10, x1, x2 (x10 = 1)
- SLTU x11, x1, x2 (x11 = 1)
- SW x3, 0(x0) (mem[0] = 15)
- LW x12, 0(x0) (x12 = 15)
- ADD x13, x12, x1 (x13 = 20)
- JAL x0, -56 (jump back to the first instruction)

5.4 SIMULATION SETUP

Two versions of the top-level module were created from the same set of core modules (Program Counter, Instruction Memory, Register File, Immediate Generator, Control Unit, ALU, Data Memory):

- `riscv_top_sim` — used for behavioral simulation, with reset (`rst`) as a plain input port and additional debug output ports for observing internal signals such as the current PC and instruction.
- `riscv_top_hw` — used for hardware implementation, with reset driven by the VIO core and only the essential `led_result` and `led_opcode` output ports, to remain within the FPGA's available I/O pin budget.

A self-checking testbench applies a reset pulse for the first three clock cycles, releases reset, allows the processor to execute the full test program, and then compares each register's final value against its expected value, printing a PASS or FAIL message for every instruction. This methodology allowed simulation bugs — for example, an incorrectly encoded operand field in one instruction — to be identified and corrected before proceeding to hardware implementation.

5.5 HARDWARE IMPLEMENTATION SETUP

5.5.1 Target FPGA

The design was implemented on a Digilent Arty A7-100T development board, which is built around the Xilinx Artix-7 `xc7a100tcs324-1` FPGA. This device provides 101,440 logic cells, considerably more than the smaller Arty A7-35T variant, and was accessed through a remote hardware laboratory maintained by the institute.

5.5.2 Pin Constraints (XDC)

A Xilinx Design Constraints (XDC) file was written to map the top module's ports to physical FPGA pins: the 100 MHz clock input to pin E3, the eight `led_result` bits to the board's four user LEDs and the green channels of the four RGB LEDs, and the four `led_opcode` bits to the red channels of the RGB LEDs. This mapping keeps the total number of constrained I/O ports well within the number of pins available on the package, avoiding the IO-placement errors encountered with an earlier, more heavily-ported version of the design.

5.5.3 VIO-Based Reset Control

Because the remote laboratory does not provide access to the physical reset button on the board, a Virtual Input/Output (VIO) IP core was added to the hardware top module. The VIO core exposes one output probe, used to drive the processor's internal reset signal from the Vivado Hardware Manager, and two input probes, used to continuously read back the `led_result` and `led_opcode` values directly from the FPGA. This allows the reset signal to be toggled and the processor's internal state to be observed remotely, entirely through the Vivado GUI over the network connection to the lab.

5.5.4 Clock Divider for LED Visibility

The FPGA's 100 MHz system clock causes the processor to complete all instructions in the test program within a few hundred nanoseconds, far too fast for the LED output of any single instruction to be visually observed. A simple clock-divider counter was therefore added to the hardware top module, generating a slow derived clock (on the order of 1 Hz) that drives the Program Counter, Register File and Data Memory, while the LED outputs themselves remain combinational functions of the current ALU result and opcode. This allows each instruction's result to remain visible on the LEDs for approximately one second before advancing to the next instruction.

5.6 SYNTHESIS AND IMPLEMENTATION REPORTS

The design was synthesized and implemented in Xilinx Vivado targeting the xc7a100tcsg324-1 device. The resource utilization, timing summary and power utilization reports generated by Vivado are shown in Figures 5.1, 5.2 and 5.3.

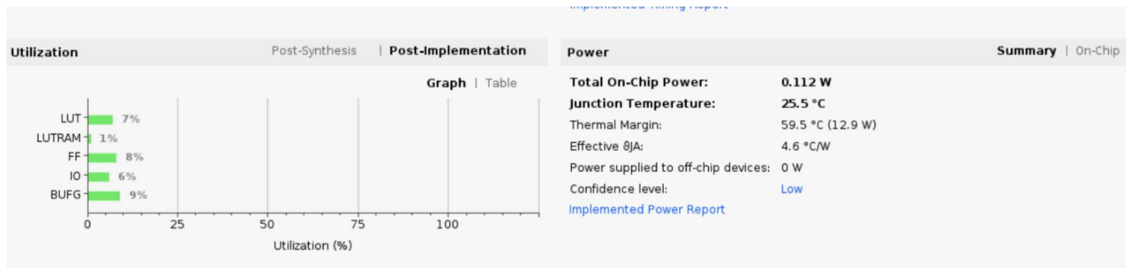


Figure 5.1 Resource Utilization summary

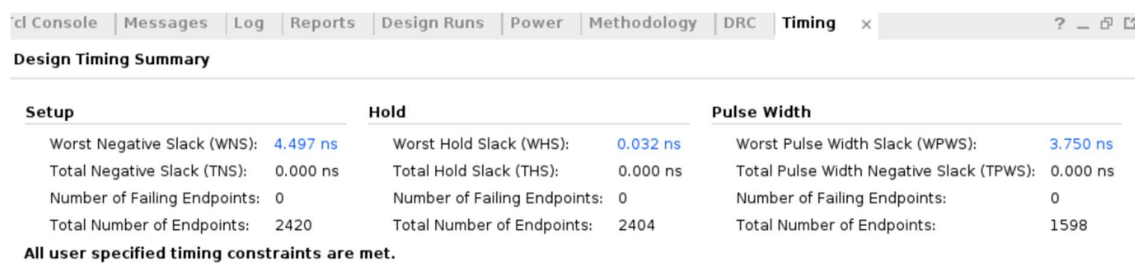


Figure 5.2 Timing summary

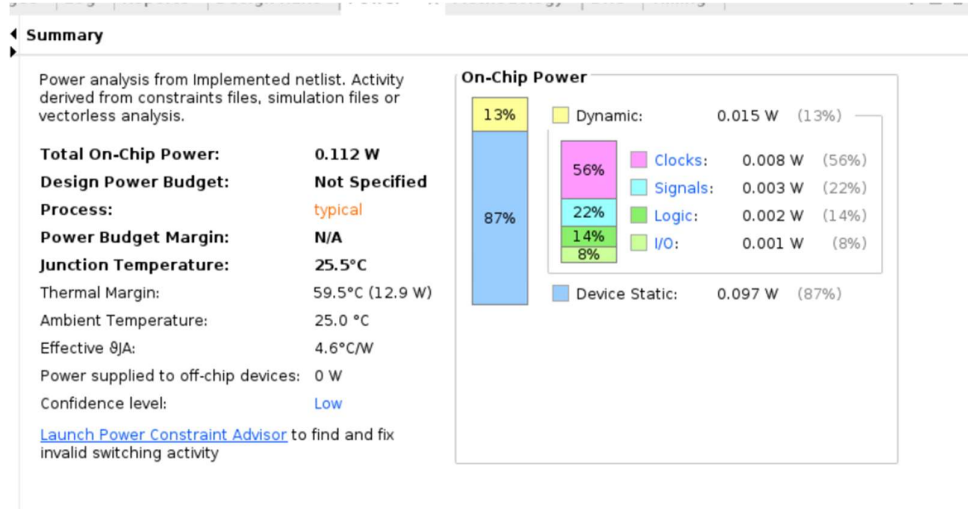


Figure 5.3 Power Utilization summary

CHAPTER 6

RESULT AND DISCUSSION

6.1 SIMULATION RESULTS

The complete processor was verified in Vivado behavioral simulation using the self-checking testbench described in section 5.4. All fourteen opcodes exercised by the test program produced the expected results, as summarized in Table 6.1.

Instruction	Register / Memory	Result	Status
ADDI	x1	5	PASS
ADDI	x2	10	PASS
ADD	x3	15	PASS
SUB	x4	5	PASS
AND	x5	0	PASS
OR	x6	15	PASS
XOR	x7	15	PASS
SLLI	x8	10	PASS
SRLI	x9	5	PASS
SLT	x10	1	PASS
SLTU	x11	1	PASS
SW	mem[0]	15	PASS
LW	x12	15	PASS
ADD	x13	20	PASS

Table 6.1 Simulation verification summary (14 opcodes)

The simulation waveform and the Tcl Console output confirming all PASS results are shown in Figures 6.1 and 6.2.

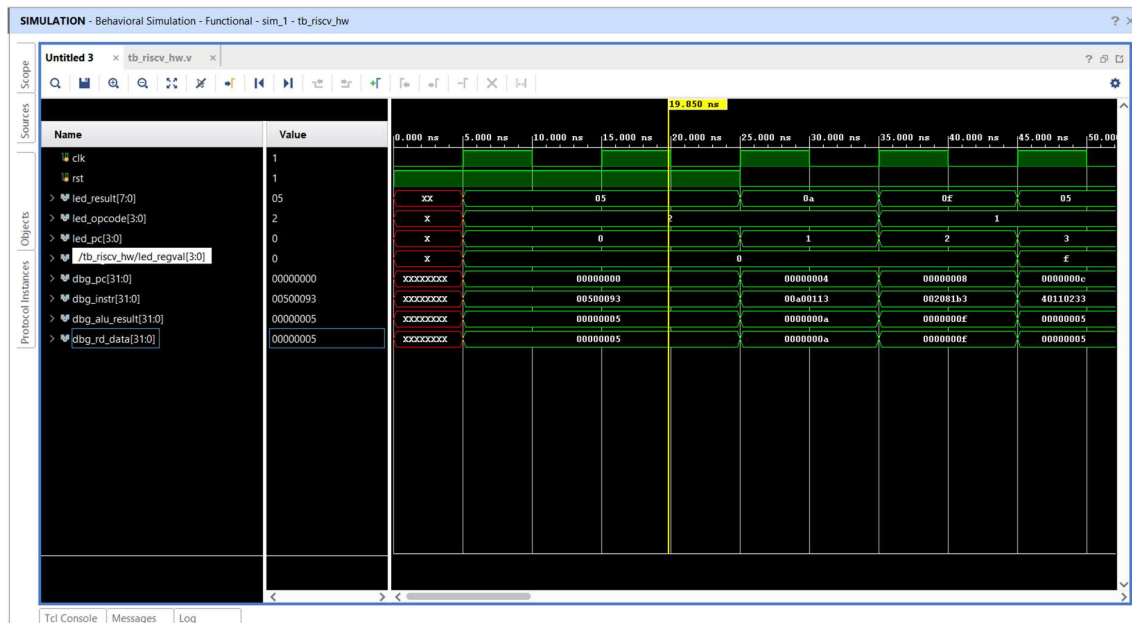


Figure 6.1 Simulation waveform showing clk, rst, PC, instruction and ALU result

```

SIMULATION - Behavioral Simulation - Functional - sim_1 - tb_riscv_hw

Tcl Console x Messages Log

# } else {
#   send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to
# }
# }
# run 1000ns
===== RISC-V Hardware Simulation =====

--- Verifying Opcodes ---
PASS | ADDI x1=5
PASS | ADDI x2=10
PASS | ADD x3=15
PASS | SUB x4=5
PASS | AND x5=0
PASS | OR x6=15
PASS | XOR x7=15
PASS | SLLI x8=10
PASS | SRLI x9=5
PASS | SLT x10=1
PASS | SLTU x11=1
PASS | SW mem[0]=15
PASS | LW x12=15
PASS | ADD x13=20 (final result before loop)

--- LED Values ---
led_result = 00000000 = 0
led_opcode = 0010
led_pc = 1010
led_regval = 1111
===== Done =====

$finish called at time : 276 ns : File "D:/VLSI SOC DESIGN/MAIN PROJECT/New folder/final/tb_riscv_hw.v" Line 69
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_riscv_hw_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns

Type a Tcl command here

```

Figure 6.2 Tcl Console showing simulation PASS results for all opcodes

6.2 HARDWARE RESULTS

Following successful simulation, the design was synthesized, implemented, and the bitstream was generated and loaded onto the Arty A7-100T FPGA through the remote hardware laboratory.

6.2.1 Bitstream Generation and Programming

The generated bitstream was downloaded onto the FPGA using the Vivado Hardware Manager connected to the remote lab's local hardware server. Figure 6.3 shows the Hardware Manager after the device was successfully programmed.

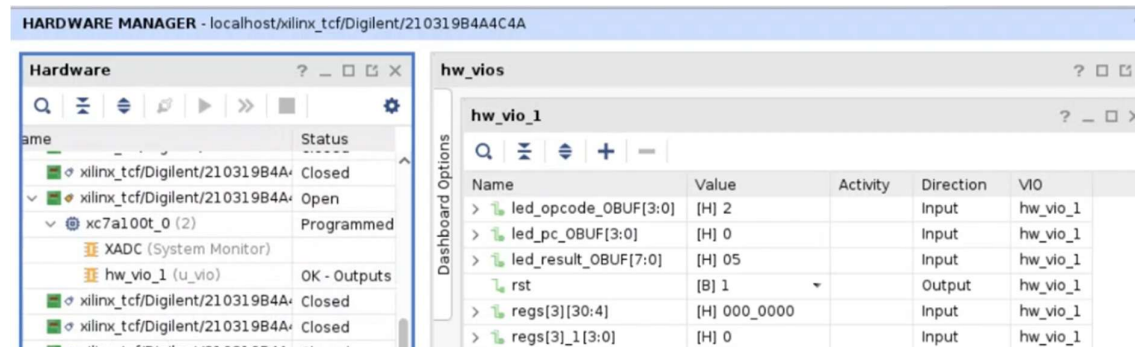


Figure 6.3 Vivado Hardware Manager – bitstream programming complete

6.2.2 LED Output on the Board

Once programmed and released from reset, the processor executes the test program continuously (looping via the final JAL instruction), and the led_result LEDs display the ALU result of the currently executing instruction in binary, while the led_opcode LEDs display the decoded instruction type. Figure 6.4 shows the physical board with the LEDs glowing during execution, captured through the remote laboratory's camera feed.

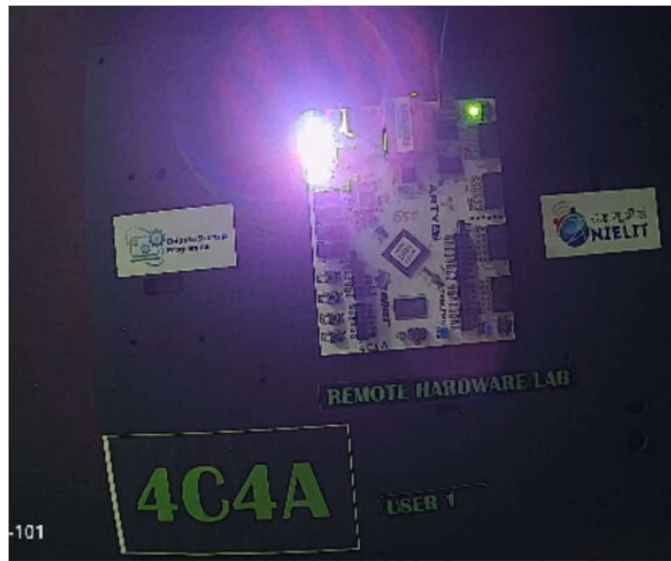


Figure 6.4 Physical board with LED output glowing during execution

6.2.3 VIO Dashboard Verification

The VIO dashboard in the Vivado Hardware Manager was used to toggle the reset signal and to read back the exact binary values of `led_result` and `led_opcode` directly from the FPGA, providing a precise, repeatable verification method in addition to the visual LED output. Figure 6.5 shows the VIO dashboard with `rst` released (`rst = 0`) and the corresponding `led_result` and `led_opcode` values read from the hardware.

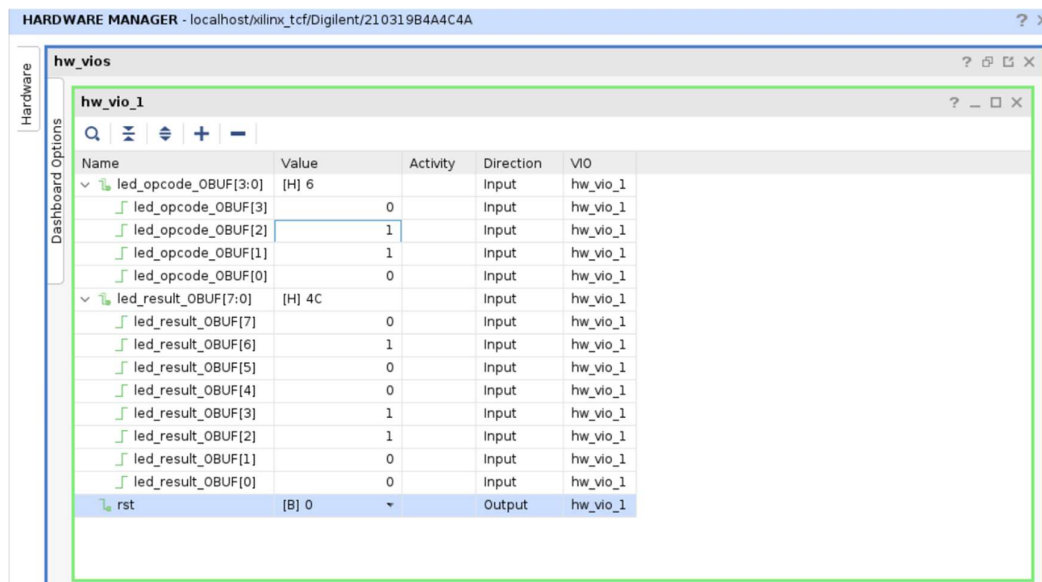


Figure 6.5 VIO dashboard showing `led_result` and `led_opcode` values read from hardware

6.3 DISCUSSION

The simulation and hardware results together confirm that the processor correctly fetches, decodes and executes all implemented opcodes. During verification, a discrepancy was observed between the expected and simulated result of the SUB instruction; tracing the machine-code encoding bit-by-bit revealed that the source-register field of that instruction had been mis-encoded, causing the ALU to subtract a register from itself instead of the intended two operands. Correcting the instruction encoding resolved the discrepancy, and re-simulation confirmed the expected result. This process reinforced the importance of verifying machine-code encodings directly against the RV32I instruction format rather than assuming a hand-written hex value is correct.

On hardware, because the VIO dashboard samples signal values at arbitrary moments while the processor runs continuously, individual snapshots may show the processor in the middle of executing the loop-back (JAL) instruction rather than one of the arithmetic instructions. This is expected behavior rather than a design fault: it reflects the very fast execution speed of the processor relative to the rate at which values can be manually sampled through the debug interface, and does not affect the correctness of execution, which is independently confirmed by both the simulation results and by the changing values of `led_result`, `led_opcode` and the program counter observed across multiple hardware readings.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

A 32-bit single-cycle RISC-V (RV32I) processor was successfully designed in Verilog HDL, verified through behavioral simulation, and implemented on a Xilinx Arty A7-100T FPGA through a remote hardware laboratory. Each of the eight core modules — Program Counter, Instruction Memory, Register File, Immediate Generator, Control Unit, ALU, Data Memory and the datapath multiplexers — was designed and verified independently before integration. A custom test program exercising fourteen distinct RV32I opcodes was used to verify the complete processor, achieving correct results for every instruction in simulation.

The design was subsequently made synthesizable and deployed on real FPGA hardware, with a Virtual Input/Output core providing remote reset control and observability, and a clock divider making the LED output visually interpretable. The processor was confirmed to execute all opcodes correctly on hardware, with results matching those obtained in simulation. This project therefore demonstrates the complete processor design flow — from RTL coding and module-level verification, through system-level simulation, to FPGA synthesis and remote hardware verification — and provides a solid practical foundation in computer architecture and digital design.

7.2 FUTURE SCOPE

Several extensions to this project could be pursued to further deepen and broaden its scope:

- **Pipelining:** Converting the single-cycle datapath into a five-stage pipelined architecture would improve instruction throughput and maximum operating frequency, at the cost of additional hazard-detection and forwarding logic.
- **Extended Instruction Support:** Adding support for remaining RV32I instructions (such as LUI, AUIPC, JALR and the remaining branch instructions) and standard extensions such as M (multiply/divide) would allow the processor to run a wider range of real-world programs.

- **Memory Hierarchy:** Introducing an instruction/data cache would allow the design to be evaluated with more realistic memory access patterns and larger programs.
- **Toolchain Integration:** Interfacing the processor with a RISC-V GCC toolchain would allow C programs to be compiled and executed directly, rather than relying on hand-assembled test programs.
- **UART-Based I/O:** Adding a UART interface would allow programs to be loaded and results to be reported over a serial link, removing the current reliance on LEDs and the VIO debug core for observing results.

REFERENCES

1. "Implementation of RISC-V Single Cycle Core," International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE), 2025.
2. "Single Cycle RISC-V Microarchitecture Processor and its FPGA Prototype," IEEE Conference Publication, IEEE Xplore.
3. "Design and FPGA Implementation of Five Stage Pipelined RISC-V Processor," IEEE Conference Publication, IEEE Xplore.
4. "Design and Implementation of a RISC-V Processor on FPGA," IEEE Conference Publication, IEEE Xplore.
5. A. Waterman and K. Asanović (Eds.), "The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture," RISC-V International.
6. Digilent Inc., "Arty A7 Reference Manual," Digilent Inc.
7. Xilinx Inc., "Vivado Design Suite User Guide: Synthesis, Implementation and Programming," AMD-Xilinx.
8. D. A. Patterson and J. L. Hennessy, "Computer Organization and Design RISC-V Edition: The Hardware Software Interface," Morgan Kaufmann.